02 线性表

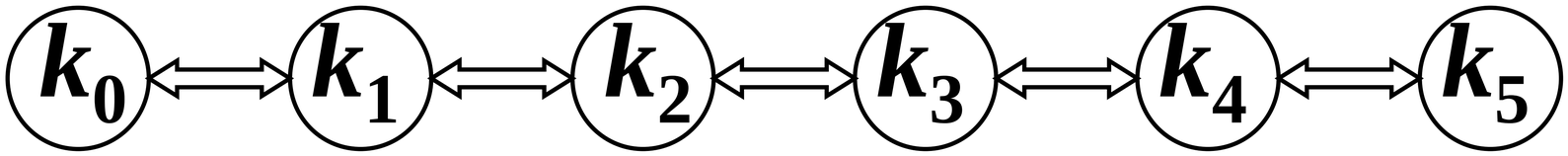
- 线性表
- 数组
- 顺序表

02 线性表

- 线性表
- 数组
- 顺序表

线性表(Linear List)

- n个数据元素的有限序列(n≥0)，记为L=(k_0, …, k_i, …, k_{n-1})
  - k_i是表L中的数据元素，k_0为表头(head)，k_{n-1}为表尾(tail)
  - n是表L的长度(若n=0，则L为空表)
  - 概念上讲，线性表中各个数据元素可以有不同的数据类型(例如，广义表)，但采用的存储表示可能会对其有限制



- 除第一个元素外，其它元素有且仅有一个直接前驱
- 除最后一个元素外，其它元素有且仅有一个直接后继

线性表示例

星期	Mon	Tue	Wed	Thu	Fri	Sat	Sun
温度	15.5	16.0	16.5	15.7	15.0	16.1	16.4

职工号	姓名	性别	年龄	工资
001	Wang	male	35	160.5
002	Cai	male	32	150.00
003	Zhang	female	28	130.00
…	…	…	…	…

线性表的操作

- 访问第i个元素($0 \le i < n$)
- 更新第i个元素的值为x($0 \le i < n$)
- 查找值为x的元素
- 插入新元素到第i个位置($0 \le i \le n$)，使原来的第i, i+1, …, n-1个元素变为第i+1, i+2, …, n个元素
- 删除第i个元素($0 \le i < n$)，使原来的第i+1, i+2, …, n-1个元素变为第i, i+1, …, n-2个元素

02 线性表

- 线性表
- **数组**
- 顺序表

一维数组(Array)

- 具有相同数据类型的n个数据元素的有限序列(n≥0)
  - 按元素的下标直接存储和访问数组元素

35	27	49	18	60	54	77	83	41	02
0	1	2	3	4	5	6	7	8	9

- 静态数组：必须在定义时指定其大小和类型
  - 例如，int array[3] = {1, 2, 3};
- 动态数组：在运行过程中才为它分配存储空间
  - 例如，int * array;

一维数组的类定义

```
const int DefaultSize = 30;
template <class Type> class Array {
   public:
      Array(int Size = DefaultSize);  // 构造函数
      Array(const Array <Type> &x);  // 复制构造函数
      ~Array( ) { delete [ ] elements; }  // 析构函数
      Array <Type> & operator = (const Array <Type> &A);  // 数组复制
      Type & operator [ ] (int i);  // 访问下标为i的数组元素
      Type * operator * ( ) const { return elements; }  // 指针转换
      int Length( ) const { return ArraySize; }  // 取数组长度
      void Resize(int sz);  // 修改数组长度
   private:
      Type *elements;  // 动态数组
      int ArraySize;  // 数组元素个数
      void GetArray( );  // 动态分配数组的存储空间
};
```

一维数组的构造函数

```
// 带形参的构造函数：建立最大长度为Size的数组
template <class Type>  Array <Type> :: Array(int Size) {
    if (Size <= 0) {  // 参数检查
        cerr<< " Invalid Array Size " <<endl;
        ArraySize = 0;
        return;
    }
    ArraySize = Size;  // 数组长度
    GetArray( );  // 创建数组
}
```

一维数组的构造函数

```
// 私有函数：动态分配数组的存储空间
template <class Type> void Array <Type> :: GetArray( ) {
    // 创建动态数组
    elements = new Type [ArraySize];
    if (elements == NULL) {
        cerr<< " Memory Allocation Error " <<endl;
        ArraySize = 0;
        return;
    }
}
```

一维数组的构造函数

```
// 复制构造函数：复制数组x以建立新数组
template <class Type>  Array <Type> :: Array(const Array <Type> &x) {
    int n = x.ArraySize; ArraySize = n;  // 目标数组的长度
    elements = new Type [n];  // 为目标数组动态分配存储空间
    if (elements == NULL) {
        cerr<< " Memory Allocation Error " <<endl;
        ArraySize = 0;  return;
    }
    Type *srcptr = x.elements;  // 源数组首地址
    Type *destptr = elements;  // 目标数组首地址
    while (n--) *destptr++ = *srcptr++;  // 复制
}
```

一维数组的访问函数

```
// 访问下标为i的数组元素
template <class Type> Type& Array <Type> :: operator [ ] (int i) {
    if (i < 0 || i > ArraySize−1) {
        cerr<< " Out of Array Bound " <<endl;
        return NULL;
    }
    return elements[i];
}
```

一维数组的修改数组长度函数

```
template <class Type> void Array <Type> :: Resize (int sz) {
    if (sz > 0 && sz != ArraySize) {
        Type * newarray = new Type[sz]; // 创建新数组
        if (newarray == NULL) {
            cerr<< " Memory Allocation Error " <<endl;
            return;
        }
        int n = (sz <= ArraySize) ? sz : ArraySize;
        Type *srcptr = elements;  // 源数组首地址
        Type *destptr = newarray;  // 目标数组首地址
        while (n--) *destptr++ = *srcptr++;  // 复制
        delete [ ] elements; elements = newarray;
        ArraySize = sz;
    }
}
```

数组的顺序存储方式

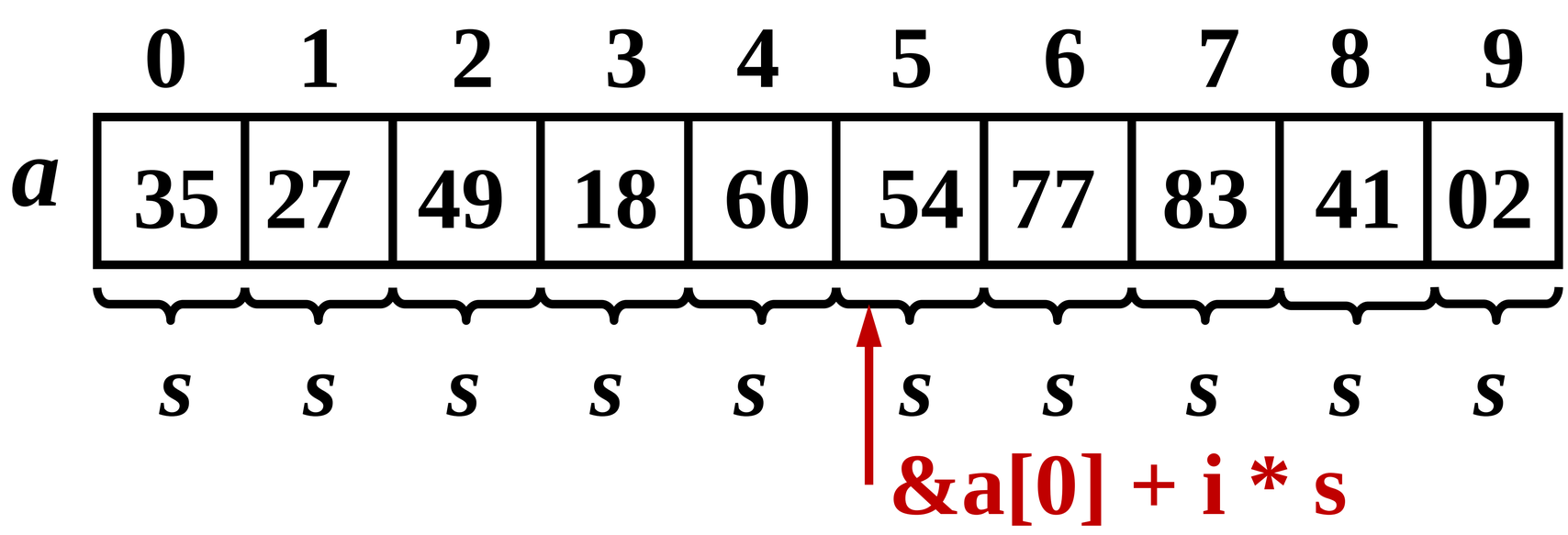
- 数组中各个元素的值是按次序存放在计算机的一组连续存储单元中

- 只要知道数组元素的下标值，就可以按相应的地址计算公式求得该元素的存放地址

- 优点在于可以**随机存取**或者**修改**数组元素的值

一维数组

$$LOC(i) = \begin{cases} \&a[0], & i = 0 \\ LOC(i-1) + s, & i > 0 \end{cases}$$



$$LOC(i) = LOC(i-1) + s = \&a[0] + i * s$$

二维数组

$$a = \begin{pmatrix} a[0][0] & a[0][1] & \cdots & a[0][t_2-1] \\ a[1][0] & a[1][1] & \cdots & a[1][t_2-1] \\ a[2][0] & a[2][1] & \cdots & a[2][t_2-1] \\ \vdots & \vdots & \ddots & \vdots \\ a[t_1-1][0] & a[t_1-1][1] & \cdots & a[t_1-1][t_2-1] \end{pmatrix}$$

若按行优先存放，且每个元素占用s个存储单元

$$LOC(i, j) = \&a[0][0] + (i * t_2 + j) * s$$

二维数组

$$a=\begin{pmatrix} a[0][0] & a[0][1] & \cdots & a[0][t_2-1] \\ a[1][0] & a[1][1] & \cdots & a[1][t_2-1] \\ a[2][0] & a[2][1] & \cdots & a[2][t_2-1] \\ \vdots & \vdots & \ddots & \vdots \\ a[t_1-1][0] & a[t_1-1][1] & \cdots & a[t_1-1][t_2-1] \end{pmatrix}$$

若按列优先存放，且每个元素占用s个存储单元

$$LOC(i, j) = \&a[0][0] + (j * t_1 + i) * s$$

三维数组

- $a[t_1][t_2][t_3]$可以看成由t_1个$t_2 * t_3$的二维数组所组成

- 若按页/行/列存放，且每个元素占用s个存储单元

$$LOC(i, j, k) = \&a[0][0][0] + (i * t_2 * t_3 + j * t_3 + k) * s$$

前i页总元素个数

第i页的前j行总元素个数

第j行前k列总元素个数

n维数组

- 各维元素个数为 $t_1, t_2, t_3, \ldots, t_n$

- 若按各维存放，且每个元素占用s个存储单元

$LOC(i_1, i_2, \ldots, i_n) = \&a[0][0]\ldots[0] +$

$(i_1 * t_2 * t_3 * \ldots * t_n + i_2 * t_3 * t_4 * \ldots * t_n + \ldots + i_{n-1} * t_n + i_n) * s$

$$= \&a[0][0]\ldots[0] + \left(\sum_{j=1}^{n-1}(ij * \prod_{k=j+1}^{n} t_k) + in\right) * s$$

练习1

- 二维数组A的每个元素是由6个字符组成，其行下标i = 0, 1, …, 8，列下标j = 1, 2, …, 10。若A按行优先存储，A[8, 5]的起始地址与当A按列优先存储时的哪个元素的起始地址相同(设每个字符占一个字节)?

- 设二维数组A[$c_1..d_1$, $c_2..d_2$]，每个元素占L个字节，则：

按行优先存储地址的计算公式：

$$LOC(i, j) = LOC(c_1, c_2) + [(i-c_1) * (d_2-c_2+1) + (j-c_2)] * L$$

按列优先存储地址的计算公式：

$$LOC(i, j) = LOC(c_1, c_2) + [(j-c_2) * (d_1-c_1+1) + (i-c_1)] * L$$

- (8-0)*(10-1+1)+(5-1)=(j-1)*(8-0+1)+(i-0)，即9(j-1)+(i-0)=84
- 推出A[3, 10]

练习2

- 三维数组M[2..3, -4..2, -1..4]中，每个元素占用2个存储单元，M的起始地址为100，如果按页优先顺序存储，M[3][-3][3]的存储地址是?

- 设三维数组A[c_1..d_1, c_2..d_2, c_3..d_3]，每个元素占L个字节，则：

页优先存储地址的计算公式：

LOC(i, j, k) = LOC(c_1, c_2, c_3) + [(i-c_1)*(d_2-c_2+1)*(d_3-c_3+1) + (j-c_2)*(d_3-c_3+1) + (k-c_3)] * L

- **LOC(3, -3, 3) = 100 + [(3-2)*(2-(-4)+1)*(4-(-1)+1) + (-3-(-4))*(4-(-1)+1) + (3-(-1))] * 2 = 100 + (42 + 6 + 4) * 2 = 204**

特殊矩阵

- 特殊矩阵是非零元素或零元素的分布有一定规律的矩阵

- 特殊矩阵的压缩存储对可以不存储的元素(如零元素或对称元素)，不再存储，从而节省存储空间，主要是针对阶数很高的特殊矩阵

- 对称矩阵、带状矩阵、稀疏矩阵

对称矩阵

- 一个n × n的矩阵A，在A中$a_{ij} = a_{ji}$

$$A = \begin{bmatrix} a_{00} & a_{01} & a_{02} & \cdots & a_{0n-1} \\ a_{10} & a_{11} & a_{12} & \cdots & a_{1n-1} \\ a_{20} & a_{21} & a_{22} & \cdots & a_{2n-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{n-10} & a_{n-11} & a_{n-12} & \cdots & a_{n-1n-1} \end{bmatrix}$$

对称矩阵的压缩存储

- 为节约存储，只存储对角线及对角线以上的元素，称为上三角矩阵；或者只存储对角线或对角线以下的元素，称为下三角矩阵

上三角矩阵

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & \cdots & a_{0n-1} \\ a_{10} & a_{11} & a_{12} & \cdots & a_{1n-1} \\ a_{20} & a_{21} & a_{22} & \cdots & a_{2n-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{n-10} & a_{n-11} & a_{n-12} & \cdots & a_{n-1n-1} \end{bmatrix}$$

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & \cdots & a_{0n-1} \\ a_{10} & a_{11} & a_{12} & \cdots & a_{1n-1} \\ a_{20} & a_{21} & a_{22} & \cdots & a_{2n-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{n-10} & a_{n-11} & a_{n-12} & \cdots & a_{n-1n-1} \end{bmatrix}$$

下三角矩阵

- 按行存放于一个一维数组B中(有n + (n-1) + … + 1 = n(n+1)/2个元素)，称之为对称矩阵A的压缩存储方式

下三角矩阵的压缩存储

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & \cdots & a_{0n-1} \\ a_{10} & a_{11} & a_{12} & \cdots & a_{1n-1} \\ a_{20} & a_{21} & a_{22} & \cdots & a_{2n-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{n-10} & a_{n-11} & a_{n-12} & \cdots & a_{n-1n-1} \end{bmatrix}$$

	0	1	2	3	4	5	6	7	8		n(n+1)/2-1
B	a_{00}	a_{10}	a_{11}	a_{20}	a_{21}	a_{22}	a_{30}	a_{31}	a_{32}	……	a_{n-1n-1}

- 若i ≥ j，A[i][j]在数组B中的存放位置为1 + 2 + … + i + j = i*(i+1)/2 + j

前i行元素总数　　第i行第j个元素前的元素个数

下三角矩阵的压缩存储

- 若$i < j$，A[i][j]在矩阵的上三角部分，在数组B中没有存放，可以找它的对称元素A[j][i] = $j*(j+1)/2 + i$

- 若已知下三角矩阵元素A[i][j]位于数组B的第k个位置，可寻找满足$i*(i+1)/2 \le k < (i+1)*(i+2)/2$的i，即为该元素的行号，而$j = k - i*(i+1)/2$，即为该元素的列号
  - 例如，当$k = 8$，$3*4/2=6 \le k < 4*5/2=10$，取$i=3$，则$j=8-3*4/2=2$

上三角矩阵的压缩存储

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & \cdots & a_{0n-1} \\ a_{10} & a_{11} & a_{12} & \cdots & a_{1n-1} \\ a_{20} & a_{21} & a_{22} & \cdots & a_{2n-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{n-10} & a_{n-11} & a_{n-12} & \cdots & a_{n-1n-1} \end{bmatrix}$$

	0	1	2		n-1	n	n+1		2n-2		n(n+1)/2-1
B	a_{00}	a_{01}	a_{02}	…	a_{0n-1}	a_{11}	a_{12}	…	a_{1n-1}	…	a_{n-1n-1}

- 若i ≤ j，A[i][j]在数组B中的存放位置为

$$n + (n-1) + (n-2) + \ldots + (n-(i-1)) + j - i = (2*n-i-1)*i/2+j$$

前i行元素总数　　第i行第j个元素前的元素个数

上三角矩阵的压缩存储

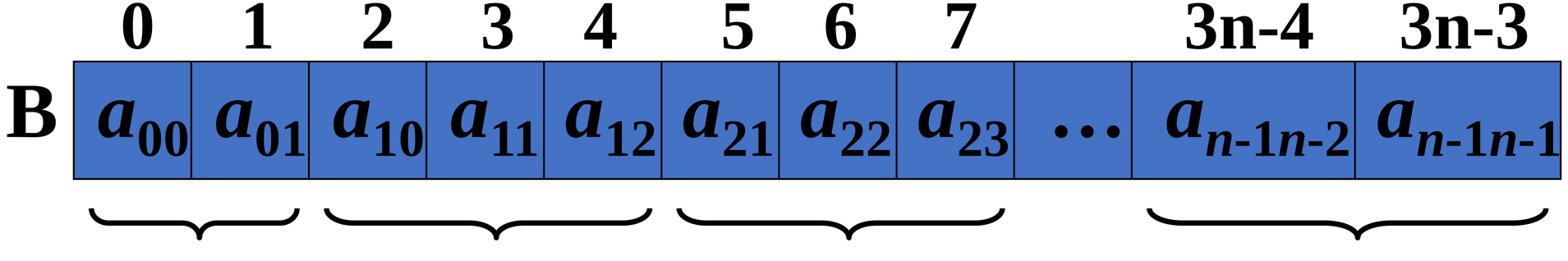
- 若i > j，A[i][j]在矩阵的下三角部分，在数组B中没有存放，可以找它的对称元素A[j][i] = (2*n-j-1)*j/2+i

带状矩阵

$$\mathrm{A}=\begin{bmatrix} a_{00} & a_{01} & 0 & 0 & 0 & 0 \\ a_{10} & a_{11} & a_{12} & 0 & 0 & 0 \\ 0 & a_{21} & a_{22} & a_{23} & 0 & 0 \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ 0 & 0 & 0 & a_{n-2n-3} & a_{n-2n-2} & a_{n-2n-1} \\ 0 & 0 & 0 & 0 & a_{n-1n-2} & a_{n-1n-1} \end{bmatrix}$$

- **三对角矩阵中除主对角线及在主对角线上下两条对角线上的元素外，所有其它元素均为0，共有3n-2个非零元素**

三对角矩阵的压缩存储



- 将三对角矩阵A中三条对角线上的元素按行存放在一维数组B中，且a_{00}存放于B[0]中
- 在三条对角线上的元素a_{ij}满足$0 \le i \le n-1, i-1 \le j \le i+1$
- A[i][j]在第i行，它前面有3*i-1个非零元素，在本行中第j列前面有j-i+1个元素，所以A[i][j]在B中位置k为2*i+j

三对角矩阵的压缩存储

- 若已知三对角矩阵元素A[i][j]位于数组B的第k个位置，可寻找满足$3i-1 \le k < 3(i+1)-1$的i，即为该元素的行号，而j = k-2i，即为该元素的列号
  - 例如，当k=8，3*3-1 ≤ 8 < 3*4-1，取i=3，则j=8-2*3=2
  - 例如，当k=10，3*3-1 ≤ 8 < 3*4-1，取i=3，则j=10-2*3=4

练习3

- 将一个A[1..100, 1..100]的三对角矩阵，按行优先存入一维数组B[1..298]中， A[66, 65](即该元素下标i=66， j=65)在B数组中的位置k为?

- 195

稀疏矩阵(Sparse Matrix)

$$A_{6\times7}=\begin{pmatrix}0&0&0&22&0&0&15\\0&11&0&0&0&17&0\\0&0&0&-6&0&0&0\\0&0&0&0&0&39&0\\91&0&0&0&0&0&0\\0&0&28&0&0&0&0\end{pmatrix}$$

- 矩阵$A_{m\times n}$中有t个非零元素，若t远远少于矩阵元素的总数m×n，则称矩阵A为**稀疏矩阵**
- 为节省存储空间，应只存储非零元素，由于非零元素的分布一般没有规律，应在存储非零元素时，同时存储该非零元素的三元组，即行下标**row**、列下标**col**、值**value**

稀疏矩阵的类定义

```
// 稀疏矩阵类的前向引用声明
template <class Type> class SparseMatrix;
// 三元组类定义
template <class Type> class Triple {
    friend class SparseMatrix <Type>;
    private:
        int row, col;  // 非零元素的行号与列号
        Type value;  // 非零元素的值
};
```

稀疏矩阵的类定义

```
template <class Type> class SparseMatrix {
    public:
        SparseMatrix (int MaxRow, int MaxCol);  // 构造函数
        int Transpose (SparseMatrix <Type> &b);  // 转置
        int Fast_Transpose (SparseMatrix <Type> &b);  // 快速转置
        SparseMatrix <Type> Multiply (SparseMatrix <Type> b);  // 相乘
    private:
        int Rows, Cols, NonZero_Terms;  // 行数、列数、非零元素数
        Triple <Type> SMArray[MaxTerms];  // 三元组表
};
```

稀疏矩阵的转置

- 一个m × n的矩阵A，其转置矩阵B是一个n × m的矩阵，且A[i][j] = B[j][i]，即矩阵A的行成为矩阵B的列，矩阵A的列成为矩阵B的行

- 用三元组表示的非零矩阵元素按行存放，当行号相同时，按列号递增的顺序存放

- 转置得到的矩阵B同样要这样存放，因此稀疏矩阵的转置运算要转化为对应三元组表的转置

稀疏矩阵的存储

稀疏矩阵

$$\begin{pmatrix} 0 & 0 & 0 & 22 & 0 & 0 & 15 \\ 0 & 11 & 0 & 0 & 0 & 17 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 39 & 0 \\ 91 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 28 & 0 & 0 & 0 & 0 \end{pmatrix}$$

	行 (row)	列 (col)	值 (value)
[0]	0	3	22
[1]	0	6	15
[2]	1	1	11
[3]	1	5	17
[4]	2	3	-6
[5]	3	5	39
[6]	4	0	91
[7]	5	2	28

转置矩阵的存储

转置矩阵

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 91 & 0 \\ 0 & 11 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 28 \\ 22 & 0 & -6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 17 & 0 & 39 & 0 & 0 \\ 15 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

	行 (row)	列 (col)	值 (value)
[0]	0	4	91
[1]	1	1	11
[2]	2	5	28
[3]	3	0	22
[4]	3	2	-6
[5]	5	1	17
[6]	5	3	39
[7]	6	0	15

用三元组表表示的稀疏矩阵及其转置

	行 (row)	列 (col)	值 (value)
[0]	0	3	22
[1]	0	6	15
[2]	1	1	11
[3]	1	5	17
[4]	2	3	-6
[5]	3	5	39
[6]	4	0	91
[7]	5	2	28

	行 (row)	列 (col)	值 (value)
[0]	0	4	91
[1]	1	1	11
[2]	2	5	28
[3]	3	0	22
[4]	3	2	-6
[5]	5	1	17
[6]	5	3	39
[7]	6	0	15

稀疏矩阵转置的算法思想

- 设矩阵列数为Cols，对矩阵三元组表扫描Cols次，第k次检测列号为k的三元组

- 第k次扫描寻找所有列号为k的三元组，将其行号变列号、列号变行号，顺次存于转置矩阵三元组表中

稀疏矩阵的转置

```
// 将矩阵a(由*this指示)转置，结果在稀疏矩阵b中
template <class Type> int SparseMatrix <Type> :: Transpose
(SparseMatrix <Type> &b) {
    int CurrentPosition;  // 存放位置指针
    b.Rows = Cols;  // 矩阵b的行数=矩阵a的列数
    b.Cols = Rows;  // 矩阵b的列数=矩阵a的行数
    // 矩阵b的非零元素个数=矩阵a的非零元素个数
    b.NonZero_Terms = NonZero_Terms;
    if (NonZero_Terms > 0) {  // 非零元素个数不为零
        CurrentPosition = 0;  // 存放位置指针清零
        for (int k = 0; k < Cols; k++)  // 按列号做扫描，做Cols趟
            for (int i = 0; i < NonZero_Terms; i++)  // 寻找列号为k的三元组
```

稀疏矩阵的转置

```
if (SMArray[i].col == k) {  // 第i个三元组中元素的列号为k
    // 新三元组的行号
    b.SMArray[CurrentPosition].row = k;
    // 新三元组的列号
    b.SMArray[CurrentPosition].col = SMArray[i].row;
    // 新三元组的值
                b.SMArray[CurrentPosition].value = SMArray[i].value;
                // 存放指针进1
                CurrentPosition++;
            }
        }
        return 0;
    }
```

时间复杂度为O(Cols*NonZero_Terms)

当NonZero_Terms与Rows*Cols等数量级时，时间复杂度为$O(Rows*Cols^2)$

稀疏矩阵快速转置的算法思想

- 为加快转置速度，建立辅助数组**rowSize**和**rowStart**
  - **rowSize**记录矩阵转置后各行(转置前各列)非零元素个数
  - **rowStart**记录各行非零元素在转置三元组表中的开始存放位置

- 扫描矩阵的三元组表，根据三元组中的列号，确定它转置后的行号，查**rowStart**表，按查到的位置直接将该项存入转置矩阵的三元组表中

稀疏矩阵快速转置的算法思想

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	语 义
rowSize	1	1	1	2	0	2	1	矩阵 A 各列非零元素个数
rowStart	0	1	2	3	5	5	7	矩阵 B 各行开始存放位置

三元组	(0)	(1)	(2)	(3)	(4)	(5)	(6)	(7)
行row	0	0	1	1	2	3	4	5
列col	3	6	1	5	3	5	0	2
值value	22	15	11	17	-6	39	91	28

稀疏矩阵的快速转置

```
template <class Type> int SparseMatrix <Type> ::
Fast_Transpose (SparseMatrix <Type> &b) {
    int *rowSize = new int [Cols]; // 统计各列非零元素个数
    int *rowStart = new int [Cols]; // 预计转置后各行存放位置
    b.Rows = Cols; b.Cols=Rows; b.NonZero_Terms=NonZero_Terms;
if (NonZero_Terms>0) {
    for (int i = 0; i < Cols; i++)  rowSize[i] = 0;
    // 根据矩阵a中非零元素的列号，统计矩阵b中第i行非零元素个数
    for (i = 0; i < NonZero_Terms; i++) rowSize[SMArray[i].col]++;
    rowStart[0] = 0;  // 计算矩阵b第i行的开始存放位置
    // rowStart[i]等于矩阵b的第i行的开始存放位置
    for (i = 1; i < Cols; i++) rowStart[i] = rowStart[i-1] + rowSize[i-1];
```

稀疏矩阵的快速转置

```
for (i = 0; i < NonZero_Terms; i++) {
    // j为第i个非零元素在b中应存放的位置
    int j = rowStart[SMArray[i].col];
    b.SMArray[j].row = SMArray[i].col;
    b.SMArray[j].col = SMArray[i].row;
    b.SMArray[j].value = SMArray[i].value;
    // 矩阵b第i行非零元素的存放位置
    rowStart[SMArray[i].col]++;
  }
 }
 delete [ ] rowSize; delete [ ] rowStart;
 return 0;
}
```

稀疏矩阵快速转置的时间复杂度

- 在函数中，共有四个循环，时间复杂度分别为O(Cols)、O(NonZero_Terms)、O(Cols)和O(NonZero_Terms)，因此该函数的时间复杂度为O(max(Cols, NonZero_Terms))
- 当NonZero_Terms与Rows*Cols等数量级时，函数的时间复杂度为O(Rows*Cols)，这与不采用三元组表表示矩阵时的转置时间复杂度相同

02 线性表

- 线性表
- 数组
- 顺序表

顺序表(Sequential List)

- 定义：将线性表中的元素相继存放在一个连续的存储空间中，可利用一维数组描述存储结构

- 特点：元素的逻辑顺序与其存放的物理顺序一致，对元素可以进行随机访问

- 限制：所有元素具有相同的数据类型

	0	1	2	3	4	5
data	25	34	57	16	48	09

顺序表的类定义

```
template <class Type> class SeqList {
    public:
        SeqList (int size=defaultSize);  // 构造函数
        ~SeqList ( ) { delete [ ] list; }  // 析构函数
        int Length ( ) const { return last+1; }  // 计算表长度
        int Find (Type &x) const;  // 查找
        int Insert (Type &x, int i);  // 插入x在表中第i个位置处
        int Remove (int i);  // 删除第i个位置处的元素
        int IsEmpty( ) { return last == -1; }  // 判断表空否
        int IsFull( ) { return last == maxSize-1; }  // 判断表满否
        Type Get (int i) { return i < 0 || i > last ? NULL : list[i]; }  // 访问第i个元素
        …
    private:
        Type *list;  // 表的存放数组
        int maxSize;  // 表的最大可容纳元素数
        int last;  // 当前已存元素的最后位置，从0开始
}
```

构造函数

```
template <class Type> SeqList <Type> :: SeqList (int size) {
    if (size > 0) {
        maxSize = size;  last = -1;  list = new Type [maxSize];
        if (list == NULL) {
            cerr<< " Memory Allocation Error " <<endl;
            return;
        }
    }
}
```

元素的查找

查找 16

0	1	2	3	4	5
25	34	57	16	48	09

i ↑

25	34	57	16	48	09

i ↑

25	34	57	16	48	09

i ↑

25	34	57	16	48	09

i ↑ 查找成功

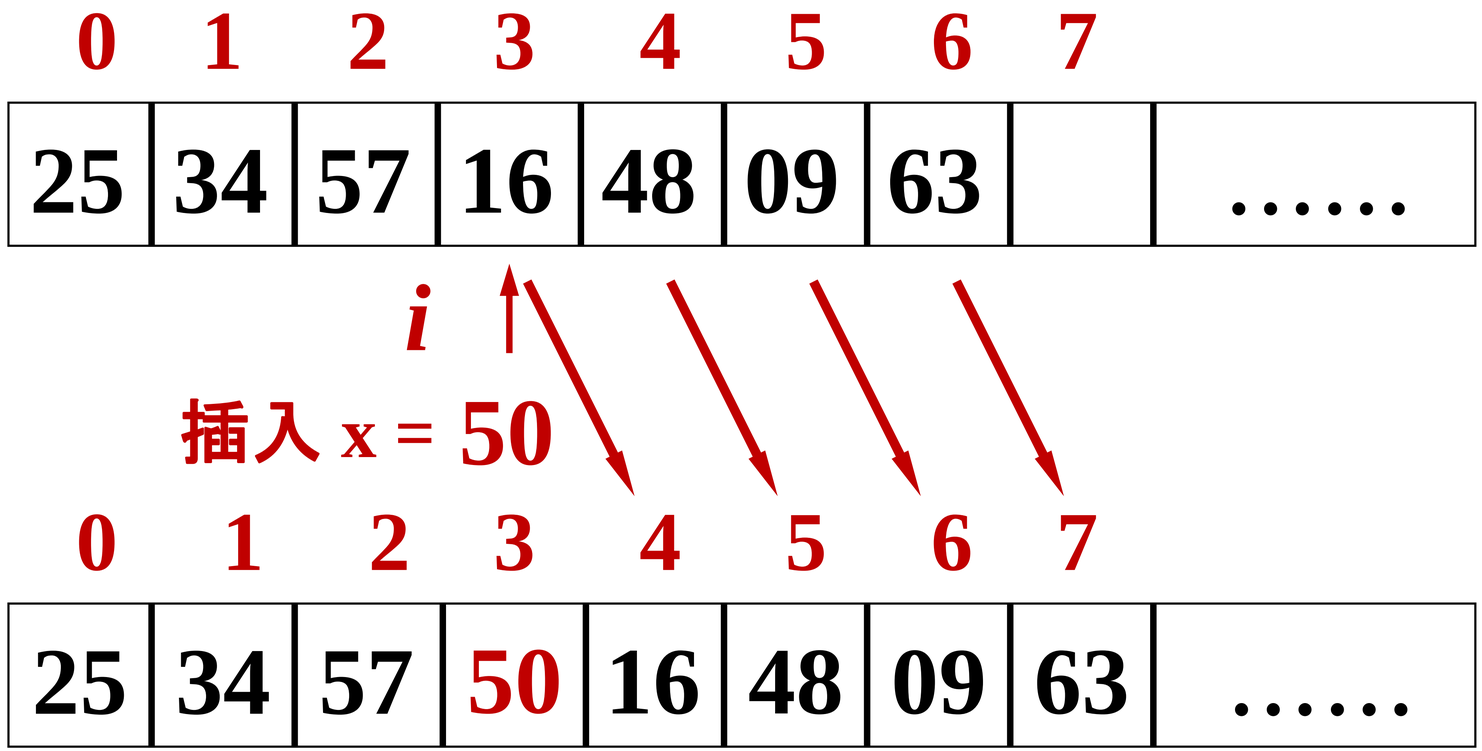
查找函数

```
// 查找函数：在表中顺序搜索与给定值x匹配的元素
template <class Type> int SeqList <Type> :: Find (Type &x) const {
// 找到则函数返回该元素的下标
// 否则函数返回-1，表示查找失败
    for (int i = 0; i <= last; i++)
        if (list[i] == x)  return i;
    return -1;
}
```

若每个元素的查找概率相等，则平均比较次数ACN $= \frac{1}{n} * \sum_{i=0}^{n-1}(i+1) = \frac{n+1}{2}$

若查找不成功，则比较次数为n

元素的插入


0 1 2 3 4 5 6 7
25 34 57 16 48 09 63 ……
i
插入 x = 50
0 1 2 3 4 5 6 7
25 34 57 50 16 48 09 63 ……

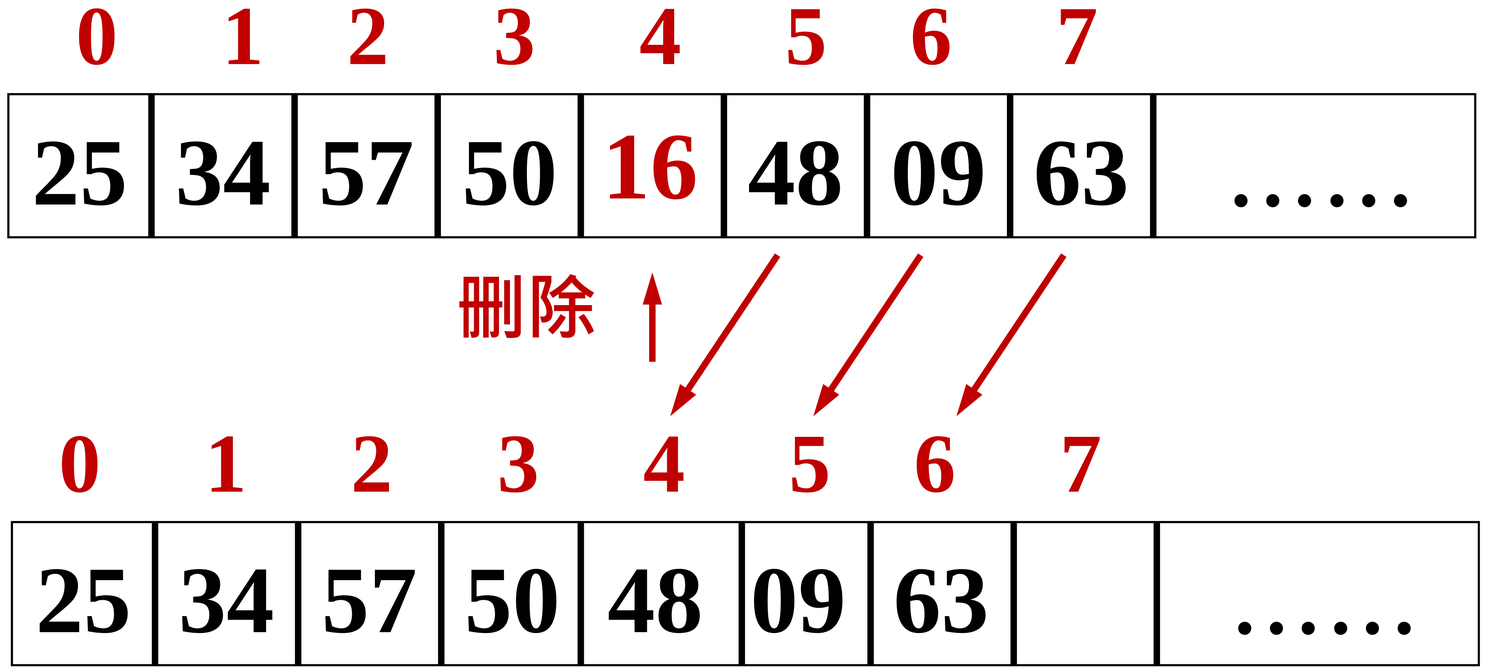
插入函数

```
// 插入x在顺序表中第i个位置处，函数返回插入是否成功的信息
template <class Type> int SeqList <Type> :: Insert (Type &x, int i) {
    if (i < 0 || i > last+1)  return 1;  // 插入位置不合理，不能插入
    if (last == maxSize-1)  return 2;  // 无可用存储单元，不能插入
    last++;  // 顺序表长度增1
    for (int j = last; j > i; j--)
        list[j] = list[j-1];  // 依次后移
    list[i] = x;  // 插入
    return 0;
}
```

若每个可插入位置的插入概率相等，

则平均移动次数AMN = $\frac{1}{n+1} * \sum_{i=0}^{n}(n-i) = \frac{n}{2}$

元素的删除



删除函数

```
// 删除第i个位置处的元素
template <class Type> int SeqList <Type> :: Remove(int i) {
    if (i < 0 || i > last)  return 1;  // 删除位置不合理，不能删除
    for (j = i; j < last; j++)
        list[j] = list[j+1];  // 依次前移
    last--;  // 表长度减1
    return 0;
}
```

若每个可删除位置的删除概率相等，

则平均移动次数AMN = $\frac{1}{n} * \sum_{i=0}^{n-1}(n-i-1) = \frac{n-1}{2}$

练习4

- 设在等概率的情况下，对有127个元素的顺序表进行插入，平均需要移动多少个元素?删除一个元素，平均需要移动多少个元素?
  - 插入时平均移动127/2=63.5个元素
  - 删除时平均移动(127-1)/2=63个元素

顺序表的应用：集合的“并”运算

```
void Union ( SeqList<int> & A, SeqList<int> & B ) {
    int n = A.Length ( );
    int m = B.Length ( );
    for ( int i = 0; i < m; i++ ) {
        int x = B.Get (i);  // 在B中取一元素
        int k = A.Find (x);  // 在A中搜索它
        if (k == −1) {  // 若未找到，则插入它
            A.Insert (x, n);
            n++;
        }
    }
}
```

顺序表的应用：集合的“交”运算

```
void Intersection (SeqList<int> & A, SeqList<int> & B ) {
    int n = A.Length ( );
    int m = B.Length ( );
    int i = 0;
    while (i < n) {
        int x = A.Get (i);  // 在A中取一元素
        int k = B.Find (x);  // 在B中搜索它
        if (k == −1) { A.Remove (i);  n−−; } // 若未找到，则删除它
        else i++;
    }
}
```

顺序表的应用：多项式(Polynomial)

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

$$= \sum_{i=0}^{n} a_i x^i$$

- n阶多项式$P_n(x)$有n+1项
  - 系数$a_0, a_1, a_2, \ldots, a_n$
  - 指数0, 1, 2, …, n，按升幂排列

多项式的类定义

```
class Polynomial {
    public:
        Polynomial( );  // 构造函数
        int operator ! ( );  // 判是否零多项式
        int LeadExp( );  // 返回最大指数
        Polynomial Add (Polynomial poly);
        Polynomial Mult (Polynomial poly);
        float Eval (float x);  // 求值
};
```

多项式的存储表示

- **第一种： 静态数组表示**

```
private:
    int degree;  // 多项式中的最高阶数
    float coef[maxDegree+1];  // 多项式的系数数组
```

- **$P_n(x)$可以表示为(设pl是Polynomial类的一个对象)**

  pl.degree = n

  pl.coef[i] = a_i, $0 \le i \le n$

- **当pl.degree远小于maxDegree，大多数元素是空的**

	0	1	2		degree	maxDegree-1
coef	a_0	a_1	a_2	……	a_n	……

多项式的存储表示

- **第二种**：动态数组表示

```
private:
    int degree;
    float *coef;

Polynomial :: Polynomial(int sz) {
    degree = sz;
    coef = new float[degree+1];
};
```

- **适用于指数连续排列的多项式，但对于指数不全的多项式(例如，$P_{101}(x)=3+5x^{50}-14x^{101}$)，造成存储空间浪费**

多项式的存储表示

- **第三种：** 同时记录各项系数和指数

```
class Polynomial;
class Term {  // 多项式的项定义
    friend Polynomial;
  private:
    float coef;  // 系数
    int exp;  // 指数
};
```

	0	1	2		i		m
coef	a_0	a_1	a_2	……	a_i	……	a_m
exp	e_0	e_1	e_2	……	e_i	……	e_m

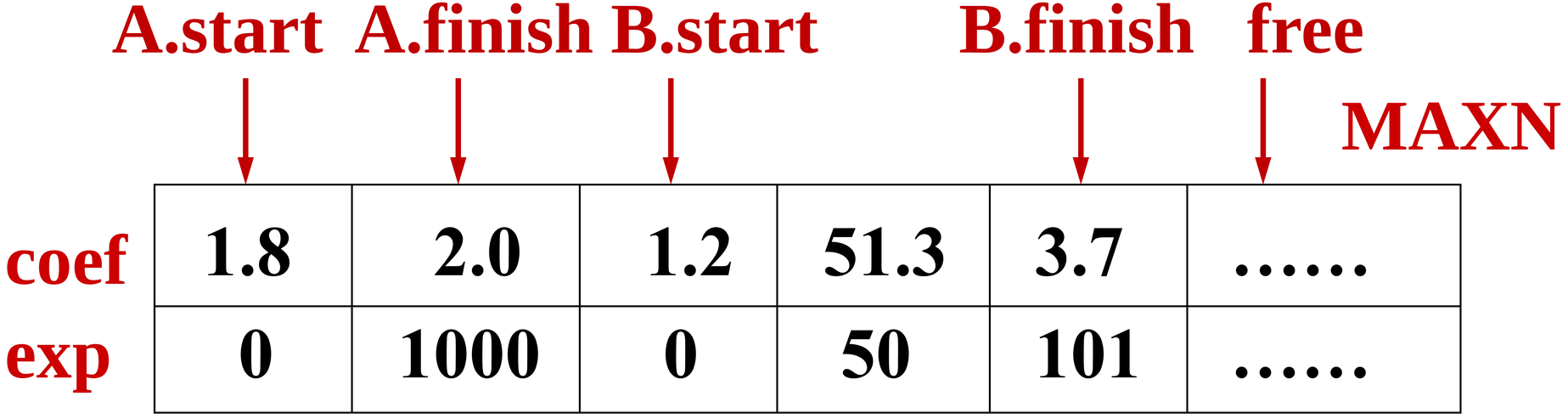
多项式的类定义

```
class Polynomial {
  public:
    Polynomial ( );  // 无形参构造函数，返回多项式A(x)=0
    // 插入系数为c、指数为e的项至指针free所指示位置
    int Poly_Append (float c, int e);
    // 返回多项式*this与多项式poly的和
    Polynomial * Poly_Add (Polynomial poly);
    ……
  private:
    static Term * Poly_Array;  // 存放多项式的数组
    static int free;  // 多项式有效项数的下一位置
    // 要求在类外初始化
    // Polynomial :: Poly_Array = new Term [MAXN+1];
    // Polynomial :: free = 0;
    int start, finish;
};
```

多项式存储示例

$A(x) = 2.0x^{1000} + 1.8$

$B(x) = 1.2 + 51.3x^{50} + 3.7x^{101}$



两个多项式存放在Poly_Array中

两个多项式相加的算法思想

- 结果多项式另存
- 扫描两个相加多项式，若都未检测完，则
  - 若当前被检测项指数相等，系数相加，若未变成0，则将结果加到结果多项式
  - 若当前被检测项指数不等，将指数小者加到结果多项式
- 若有一个多项式已检测完，将另一个多项式剩余部分复制到结果多项式

多项式增加项

```
int Polynomial :: Poly_Append (float c, int e) {
    if (free > MAXN)  // 无可用空闲存储单元，不能插入
        return 1;
    Poly_Array[free].coef = c;
    Poly_Array[free].exp = e;
    free++;
    return 0;
}
```

多项式加法

```
Polynomial * Polynomial :: Poly_Add(Polynomial B) {
    Polynomial * C = new Polynomial();  // 结果多项式
    int ah = start, bh = B.start;  // ah与bh分别为两个多项式的检测指针
    C.start = free;
    float c;
while (ah <= finish && bh <= B.finish)
    switch(comp(Poly_Array[ah].exp, B.Poly_Array[bh].exp)) { // 比较对应项指数
        case '= ': c = Poly_Array[ah].coef + Poly_Array[bh].coef;  // 相等，系数相加
                   if (c) Poly_Append (c, Poly_Array[ah].exp);  // 和非零，加新项
                   ah++;  bh++;
                   break;
```

多项式加法

```
case '<': Poly_Append (Poly_Array[ah].coef, Poly_Array[ah].exp);
          ah++;
          break;
case '>': Poly_Append (Poly_Array[bh].coef, Poly_Array[bh].exp);
          bh++;
      }
   for ( ; ah <= finish; ah++)  // 将A(x)中剩余项加入结果多项式中
      Poly_Append (Poly_Array[ah].coef , Poly_Array[ah].exp);
   for ( ; bh <= B.finish; bh++)  // 将B(x)中剩余的项加入结果多项式中
      Poly_Append (Poly_Array[bh].coef, Poly_Array[bh].exp);
   C.finish = free-1;
   return C;
}
```

多项式加法的时间复杂度分析

- **函数Poly_Append的时间复杂度是O(1)**
- **在函数Poly_Add中，三个循环之外的语句的时间复杂度也是O(1)，而三个循环的时间复杂度为O(m+n)**
- **Poly_Add的时间复杂度为O(m+n)**

本章小结

- 线性表
- 数组
- 顺序表

作业

(1) • 五对角矩阵中k与i、j的关系

P130

$$k=\begin{cases}4i+j, & i=0\\ 4i+j-1, & 0<i<n-1\\ 4i+j-2, & i=n-1\end{cases}$$

(2) • 编写一个程序，以不多于3n/2的平均比较次数，在一个有n个整数的顺序表中找出最大值和最小值的整数，要求使用的附加空间尽量少

(3) • 编写一个程序，从顺序表中删除具有给定值x的所有元素

(4) • 编写一个程序，将顺序表中的元素进行原地逆置，即($a_0, a_1, \ldots, a_{n-2}, a_{n-1}$)置换为($a_{n-1}, a_{n-2}, \ldots, a_1, a_0$)